

32-Bit Pipelined ALU Project Description

Verilog RTL design with top-level control, register file, pipeline registers, ALU core, and writeback path

Project Overview

This project implements a synthesizable 32-bit pipelined Arithmetic Logic Unit (ALU) in Verilog. The design follows RTL design practice and is organized in a modular way so that each hardware block has a clear role. The complete system uses a control unit, a register file, pipeline registers, a 32-bit ALU core, and a multiplexer for selecting the value that is written back into the register file.

The main goal of the project is to demonstrate how data moves through a hardware pipeline while the control path keeps the opcode, read addresses, write address, and write enable aligned with the correct data.

Top-Level Architecture

- The top-level module instantiates all submodules and connects them using wires.
- The control unit decodes the 16-bit instruction and generates the 4-bit opcode, read addresses, write address, write enable, load control, and clear control.
- The register file stores 16 words of 32 bits each and supplies two read outputs to the pipeline.
- The pipeline registers pass operand data from one stage to the next on the clock edge.
- The ALU performs arithmetic, logical, shift, inversion, and compare operations.
- The final output path uses a multiplexer so that either the ALU result or an external 32-bit input can be written back.

Instruction Format

The instruction word is 16 bits wide. It is split into four 4-bit fields:

- [15:12] - opcode
- [11:8] - read address 1
- [7:4] - read address 2
- [3:0] - write address

This format keeps the design compact and easy to decode. The same instruction word is enough to select the operation, choose the source registers, and identify the destination register.

Data Flow Through the Pipeline

1. Stage 1: The instruction is decoded and the addresses are captured.
2. Stage 2: The register file supplies operand data to the pipeline registers.
3. Stage 3: The ALU computes the selected operation and generates status flags.
4. Stage 4: The result is routed through the writeback multiplexer.
5. Stage 5: The final value is written into the selected register location.

Because the design is pipelined, different instructions can be in different stages at the same time. While one instruction is being written back, another can already be in the execute stage and a third can be in the read stage. This improves throughput and gives the design a more realistic processor-like structure.

Main Modules

- **CONTROL_UNIT**: Decodes the instruction and generates control and address signals.
- **REG_FILE**: Stores 16 registers of 32 bits and supports two read ports and one write port.
- **P_REGISTER**: Holds pipeline data between stages on the rising clock edge.
- **ALU_32_BIT**: Performs the actual 32-bit computation and produces flags.
- **MUX**: Selects between the ALU result and the external input value for writeback.
- **PIPELINED_ALU**: The top-level integration module.

Verification Strategy

- Each important module is verified individually with a SystemVerilog testbench.
- The top-level module is verified after integration of all blocks.
- Waveform checking is used to confirm that addresses, operands, results, and flags stay aligned.
- Edge cases such as zero, all ones, overflow, and compare conditions are included in testing.
- Back-to-back instruction sequences are used to confirm that the pipeline works correctly.

Result

The final result is a clean RTL implementation of a 32-bit pipelined ALU that can be expanded later with more instructions, hazard handling, stronger control logic, or additional processor-style blocks. The design is suitable for learning and future digital design projects.